

SMART INDIA HACKATHON 2026

MoSPI · PAIMANA · Data Informatics & Innovation Division

SOFTWARE REQUIREMENTS SPECIFICATION & COMPLETE SYSTEM ARCHITECTURE

AI-Powered Predictive Infrastructure Monitoring & Early-Warning System

PAIMANA Platform — SIH 2026 Technical Proposal

Attribute	Detail
Document Version	1.0 — SIH 2026 Submission
Problem Statement	PAIMANA / MoSPI — Predictive Infrastructure Monitoring
Ministry	Ministry of Statistics & Programme Implementation (MoSPI)
Division	Data Informatics & Innovation Division (DIID)
Date	August 2026
Tech Stack	Python · FastAPI · PostgreSQL · React · scikit-learn · XGBoost · LightGBM · SHAP · Ollama
License	Open-Source (MIT) — per MoSPI requirement

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, non-functional, and architectural requirements for PAIMANA-AI — an AI-powered predictive analytics and early-warning system designed to reduce cost and time overruns on India's public infrastructure projects. The document is intended for SIH 2026 judges, MoSPI technical reviewers, the development team, and future MoSPI integration engineers.

1.2 Project Background

India's PAIMANA (Project Assessment & Integrated Monitoring & Networking Application) portal, managed by MoSPI's IPMD/DIID, tracks approximately 1,981 active central sector projects with an original cost of ₹150 crore or above. Historical data from the OCMS system spans nearly two decades. Despite this rich data asset, projects routinely suffer cost and schedule overruns — a systemic challenge documented across all 10 major infrastructure-spending nations reviewed in the accompanying Global Comparative Analysis.

Three root causes recur worldwide: (a) optimism bias at the appraisal stage, (b) lack of continuous, data-driven monitoring during execution, and (c) absence of independent governance gates that wire early-warning signals to consequential decisions. PAIMANA-AI addresses all three.

1.3 Scope

PAIMANA-AI is a full-stack, open-source web platform that:

- Ingests and validates monthly CUF (Common Update Format) submissions from implementing agencies
- Benchmarks every project against a reference class of historically similar completed projects (UK/Netherlands RCF model)
- Predicts cost-overrun probability and schedule-delay risk using an ensemble ML pipeline (Random Forest, XGBoost, LightGBM) benchmarked against EVM/statistical baselines
- Generates SHAP-based natural-language explanations for every risk score
- Publishes a per-project Confidence Index (Australia model) on a role-based dashboard
- Auto-routes high-risk projects to a defined governance review workflow
- Includes two novel AI features: (a) Narrative Inconsistency Detector using LLM analysis, and (b) Peer-Pressure Benchmarking Engine for inter-agency intelligence

1.4 Definitions & Abbreviations

Term	Definition
CUF	Common Update Format — the monthly project-data submission form used by implementing agencies
OCMS	Online Computerised Monitoring System — the predecessor to PAIMANA, holding ~20 years of historical data
IPMD	Infrastructure and Project Monitoring Division, MoSPI
DIID	Data Informatics & Innovation Division, MoSPI
RCF	Reference Class Forecasting — statistical method comparing a project to similar completed peers
EVM	Earned Value Management — traditional cost/schedule performance tracking methodology
SHAP	SHapley Additive exPlanations — model-agnostic ML explainability framework
P50/P80/P90	Percentile estimates: 50%, 80%, 90% probability of not exceeding the forecast value
DPR	Detailed Project Report — the primary project planning document

NID	Narrative Inconsistency Detector — novel feature described in Section 10
PBE	Peer-Pressure Benchmarking Engine — novel feature described in Section 11
MVP	Minimum Viable Product — the SIH hackathon deliverable

2. Overall System Description

2.1 Product Perspective

PAIMANA-AI sits as an intelligent analytics layer on top of the existing PAIMANA data infrastructure. It does not replace the CUF submission workflow — it enriches it. Data flows from agencies → PAIMANA portal → PAIMANA-AI API → ML pipeline → dashboard. The system is entirely open-source and deployable on MoSPI's on-premise or government-cloud infrastructure.

2.2 User Classes

User Class	Description	Primary Actions
IPMD Officer	Central monitoring authority	Review risk alerts, trigger governance actions, access all projects
Ministry Official	Senior policymaker	View aggregated national risk dashboard, confidence index per sector
Implementing Agency	Project executing body	Submit/update CUF data, view own project risk score & explanation
Data Analyst (DIID)	MoSPI analytics team	Run model experiments, access raw feature importances, manage reference classes
Public / Auditor	Transparency stakeholders	Read-only access to aggregated sector/state risk summaries

2.3 Constraints

- All components must be open-source (MoSPI explicit requirement)
- No proprietary cloud vendor lock-in; deployable on NIC or MeghRaj government cloud
- Data residency: all project data must remain within Government of India infrastructure
- The system must never autonomously approve, reject, or reallocate project funding
- Any ML accuracy figure presented in the UI must be accompanied by a confidence/calibration indicator

3. Functional Requirements

3.1 Data Ingestion & Validation Module (FR-01 to FR-05)

ID	Requirement	Priority
FR-01	System shall ingest monthly CUF submissions via REST API, accepting JSON payloads mapping to all defined CUF fields	P0 — MVP
FR-02	System shall perform schema validation on every submission: reject missing mandatory fields, flag anomalous numeric ranges, check date consistency	P0 — MVP
FR-03	System shall compute a Data Confidence Score (DCS) per project per reporting period comprising: completeness %, freshness (days since last update), internal consistency checks, and historical reliability of the submitting agency	P0 — MVP
FR-04	System shall maintain a full versioned history of all CUF submissions so time-aware model training is possible without temporal leakage	P0 — MVP
FR-05	System shall support bulk historical import from OCMS CSV exports for model training purposes	P0 — MVP

3.2 Reference-Class Benchmarking Module (FR-06 to FR-10)

Implements the UK Reference Class Forecasting approach on PAIMANA's own historical dataset.

ID	Requirement	Priority
FR-06	System shall cluster historical and active PAIMANA projects into reference classes defined by: sector (22 MoSPI sectors), size band (₹150–500 Cr / ₹500–2000 Cr / ₹2000+ Cr), and region (state/union territory)	P0 — MVP
FR-07	System shall compute empirical cost-overrun and schedule-delay distributions per reference class from all completed projects in that class	P0 — MVP
FR-08	System shall score every new/updated CUF submission against its reference-class distribution and report: deviation from median, percentile rank within class, implied P50/P80/P90 final-cost and completion-date estimates	P0 — MVP
FR-09	System shall flag projects whose current trajectory puts them in the top 20% of overrun risk for their reference class	P0 — MVP
FR-10	System shall auto-update reference-class distributions as new completed-project records are added, ensuring continuous recalibration	P1 — near-term

3.3 Predictive ML Pipeline (FR-11 to FR-18)

ID	Requirement	Priority
FR-11	System shall train and maintain the following models, benchmarked head-to-head: (a) Naive sector-average baseline, (b) Logistic/linear regression, (c) EVM-derived statistical model, (d) Random Forest, (e) XGBoost, (f) LightGBM, (g) Optional ANN/DNN if dataset exceeds 5,000 labelled examples	P0 — MVP
FR-12	System shall perform cost-overrun CLASSIFICATION (will final cost exceed sanctioned cost by >10%?) and REGRESSION (predict final cost quantum)	P0 — MVP
FR-13	System shall perform schedule-delay CLASSIFICATION (will project complete >6 months late?) and REGRESSION (predict completion month)	P0 — MVP

FR-14	System shall implement time-aware train/validation/test splits using chronological ordering to prevent temporal data leakage	P0 — MVP
FR-15	System shall report model performance separately by sector, size band, and region — not only as a single pooled figure	P0 — MVP
FR-16	System shall expose an early-warning prediction: given data available at month N, predict overrun risk by scheduled completion, scored separately from final-outcome prediction	P0 — MVP
FR-17	System shall use SHAP (SHapley Additive exPlanations) to produce a ranked list of top-5 risk drivers for every project prediction	P0 — MVP
FR-18	System shall re-train models on a scheduled cadence (monthly recommended) to detect and correct for concept drift	P1 — near-term

3.4 Composite Risk Score & Confidence Index (FR-19 to FR-23)

ID	Requirement	Priority
FR-19	System shall combine Cost Risk, Schedule Risk, Progress Anomaly, and Governance Risk into a composite Project Risk Score, with each component individually visible	P0 — MVP
FR-20	System shall classify each project into risk categories: LOW / MODERATE / HIGH / VERY HIGH / CRITICAL, with category thresholds calibrated against the historical PAIMANA distribution (not arbitrary percentages)	P0 — MVP
FR-21	System shall display a per-project Confidence Index (the Data Confidence Score from FR-03) alongside the risk score, so reviewers can distinguish high-risk-high-confidence from high-risk-low-confidence flags	P0 — MVP
FR-22	System shall produce probabilistic P50/P80/P90 final-cost and completion-date estimates derived from reference-class distributions (FR-08), extended to probability-of-overrun outputs	P0 — MVP
FR-23	System shall publish an aggregated sector-level and state-level risk dashboard visible to Ministry-level users	P0 — MVP

3.5 Anomaly Detection Module (FR-24 to FR-27)

ID	Requirement	Priority
FR-24	System shall flag projects where financial expenditure significantly outpaces reported physical progress (threshold calibrated against PAIMANA noise levels)	P0 — MVP
FR-25	System shall flag projects where reported progress speed significantly exceeds the reference-class baseline for similar projects at the same stage (unusually fast progress can indicate misreporting)	P0 — MVP
FR-26	System shall flag sudden cost escalations inconsistent with the project's historical monthly expenditure trend	P0 — MVP
FR-27	System shall flag repeated milestone-date shifts across consecutive reporting periods	P0 — MVP

3.6 Governance & Escalation Workflow (FR-28 to FR-32)

ID	Requirement	Priority
FR-28	System shall auto-route any project crossing the HIGH risk threshold to a structured IPMD review queue (mirroring US PMOC/GAO escalation model)	P0 — MVP

FR-29	System shall enforce governance gates at key lifecycle stages: before DPR approval, before financial closure, and when physical progress crosses 50% — refusing to advance gate status if risk score is VERY HIGH or CRITICAL (Netherlands MIRT model)	P1 — near-term
FR-30	System shall record every governance action taken (review initiated, overridden, deferred) with timestamp and user ID for audit trail	P0 — MVP
FR-31	System shall record actual final cost and completion date for completed projects and use these as ground-truth labels for model recalibration	P1 — near-term
FR-32	System shall NEVER autonomously approve, reject, or reallocate project funding — all consequential decisions require an authorised human reviewer	P0 — MVP (design constraint)

4. Novel Innovative Features

Why These Two Features?

The two features below go beyond what any of the ten countries reviewed has implemented. They are (a) buildable with open-source tools in hackathon timeframes, (b) grounded in documented failure modes from the global comparative analysis, and (c) directly address the two hardest unsolved problems in public infrastructure monitoring: strategic misreporting and the absence of peer-group accountability.

4.1 — INNOVATIVE FEATURE 1: Narrative Inconsistency Detector (NID)

★NOVEL MVP-FEASIBLE No country has this

4.1.1 Problem It Solves

Every global system reviewed identified strategic misreporting — implementing agencies intentionally over-reporting progress or under-reporting delays — as a core failure mode that no purely numerical ML model can detect. Japan, China, and the Gulf use physical sensors and drone imagery as an independent check. PAIMANA cannot mandate hardware. But PAIMANA already collects one under-used signal: free-text narrative remarks that agencies submit alongside the quantitative CUF fields.

4.1.2 How It Works

The NID uses a locally-hosted open LLM (Ollama with Mistral-7B or LLaMA-3-8B — zero external API calls, full data residency) to:

- Extract factual claims from the narrative text field of each CUF submission (e.g. "land acquisition completed in March", "contractor mobilised")
- Cross-check each extracted claim against the corresponding quantitative CUF fields (e.g. is land-acquisition status field = "completed"? Does expenditure reflect contractor mobilisation?)
- Score each submission on a Narrative-Quantitative Coherence (NQC) score from 0–100
- Flag specific contradictions in human-readable form: "Narrative claims bridge pier work completed but physical progress field unchanged at 42% for 3 consecutive months"
- Track NQC scores over time per agency to identify systematic patterns (an agency that repeatedly scores low is more likely to be misreporting than one that had a single low score)

4.1.3 Technical Implementation

Component	Technology	Detail
LLM Runtime	Ollama (local)	Mistral-7B-Instruct or LLaMA-3-8B — runs on CPU, no GPU required at inference scale of ~2000 projects/month
Claim Extraction	Structured LLM prompt	Zero-shot prompt engineering extracts (claim, field_reference, expected_value) triples from narrative text
Cross-check Engine	Python rule engine	Compares extracted expected_value against CUF quantitative fields; flags delta > threshold
NQC Score	Weighted average	Per-claim coherence scores weighted by claim specificity; agency-level trend tracked in PostgreSQL
Dashboard	React component	NQC gauge per project; flagged contradictions displayed as expandable cards with LLM-generated explanation

4.1.4 Worked Example

NID Example — Flagged Contradiction

Agency: NHAI | Project: NH-44 Bypass | Reporting Period: June 2026

Narrative field: "Earthwork and subgrade preparation completed across all 4 packages.
Bridge construction at CH 12+400 commenced. Contractor resources fully deployed."

CUF Quantitative Fields:

- Physical progress: 38% (unchanged from May 2026)
- Expenditure this month: ₹4.2 Cr (vs. ₹18 Cr average for this project type at this stage)
- Contractor mobilisation status: "Partial"

NID FLAGS:

- [HIGH] Narrative claims bridge construction commenced — physical progress unchanged.
- [MEDIUM] Narrative claims full contractor deployment — CUF field = "Partial".
- [HIGH] Narrative claims earthwork completed — expenditure 77% below reference-class average.

NQC Score: 31 / 100 | Agency NQC Trend: 3-month declining | Routed to IPMD review queue.

4.2 — INNOVATIVE FEATURE 2: Peer-Pressure Benchmarking Engine (PBE)

★NOVEL MVP-FEASIBLE No country has this

4.2.1 Problem It Solves

Australia's Confidence Index and South Korea's PFS both create external accountability by making project credibility visible to parties other than the project sponsor. But they operate at the appraisal stage only, on point-in-time snapshots. No system reviewed provides dynamic, continuous, peer-group accountability — where each agency can see in real time how their project's risk trajectory compares to peers managing similar projects, and where that comparison is automatically surfaced to oversight authorities.

The PBE transforms PAIMANA's 1,981-project database from a static reporting archive into a live performance intelligence network. It creates a behavioural accountability mechanism: agencies that see peer projects completing on-budget and on-schedule face a natural pressure to investigate and explain their own deviations, before an IPMD review is triggered.

4.2.2 How It Works

- For every active project, PBE identifies its "peer cohort" — the 10–30 most similar active/recently-completed projects by sector, size band, region, implementation agency type, and construction stage
- PBE computes a Peer-Relative Performance Index (PPI): how does this project's current cost-overrun trajectory, schedule variance, and reporting quality compare to its cohort's at the same project stage?
- PBE generates a ranked Peer League Table visible to implementing agency users — showing their project's PPI percentile within its cohort, with anonymised peer comparators
- For IPMD/Ministry users, PBE surfaces Agency Performance Intelligence: which agencies are systematically underperforming their peers across their entire portfolio, signalling capacity or governance issues rather than project-specific ones
- PBE feeds the ML model as a feature: a project outperforming its peer cohort in early stages is a weaker overrun signal; one underperforming peers despite similar conditions is a stronger signal than raw numbers alone would show

4.2.3 Technical Implementation

Component	Technology	Detail
Cohort Identification	cosine similarity on feature vectors	Project features (sector, size, region, agency type, stage) embedded in a vector; k-nearest neighbours using scikit-learn NearestNeighbors

PPI Computation	Stage-normalised Z-score	For each metric, compute Z-score relative to cohort distribution at the same project-lifecycle stage; composite PPI = weighted average of per-metric Z-scores
Peer League Table	React DataGrid + Recharts	Anonymised cohort displayed as scatter plot with this project highlighted; agency user sees own absolute rank and percentile
Agency Portfolio Intelligence	Aggregated SQL + Python	Agency-level PPI aggregated across all their active projects; outlier agencies surface to IPMD dashboard
ML Feature Integration	Pandas feature engineering	Cohort-relative features (cost_variance_vs_cohort_median, schedule_variance_vs_cohort_p75) fed as additional features to XGBoost/LightGBM pipeline

4.2.4 Why This Is Genuinely Innovative

The PBE is the only mechanism in any reviewed system that creates a continuous, dynamic, data-driven peer accountability loop during project execution — not just at appraisal or at governance gates. It converts PAIMANA from a passive monitoring registry into an active performance-intelligence platform. The peer-comparison psychology effect (well-documented in behavioural economics) is particularly strong in public-sector contexts where agencies are sensitive to comparative performance visibility.

5. Non-Functional Requirements

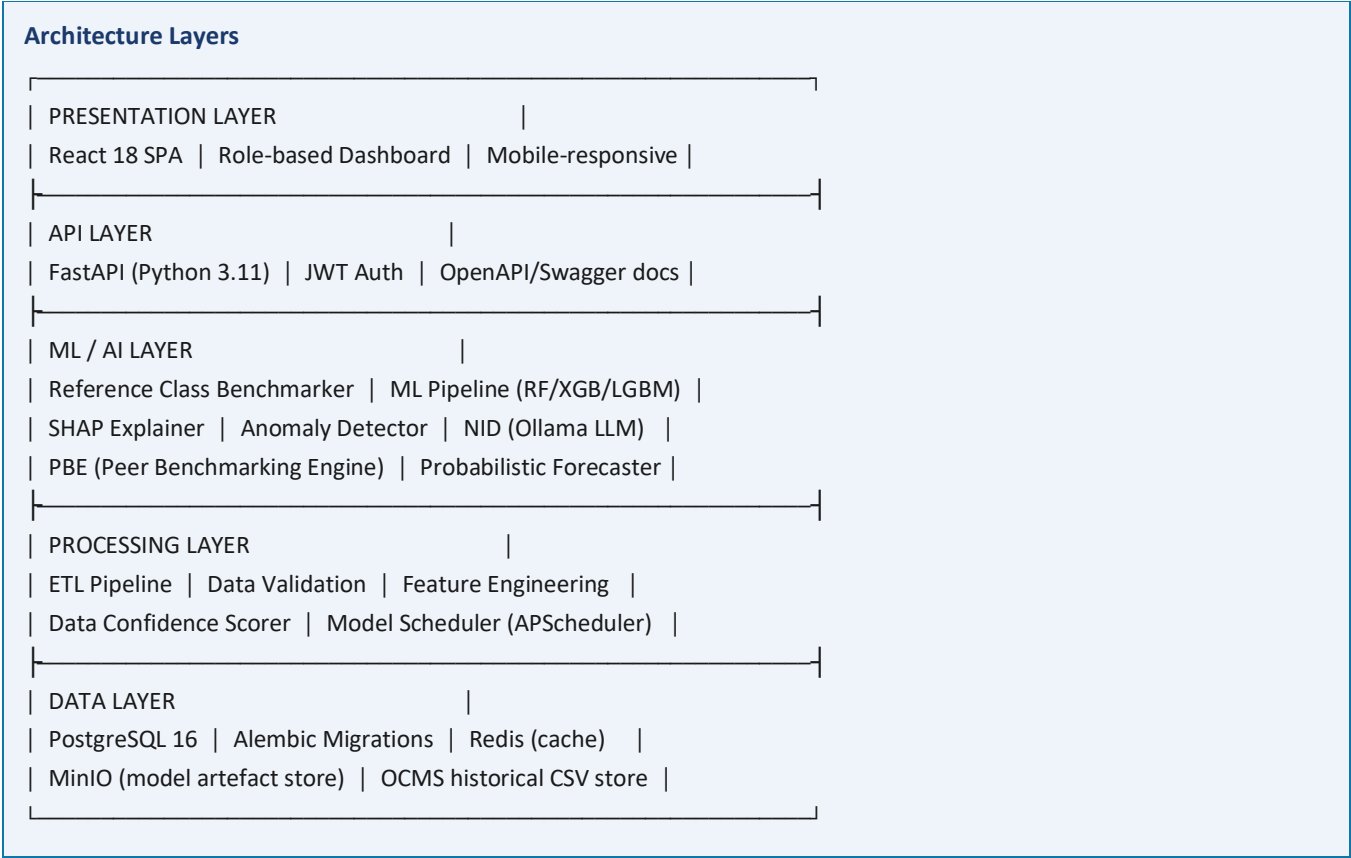
Category	Requirement	Target / Metric
Performance	API response time for risk score retrieval	< 2 seconds per project (cached scores)
Performance	Batch ML re-scoring of all 1,981 projects	< 30 minutes on a 4-core, 16GB RAM server
Performance	Dashboard load time (national view)	< 3 seconds on 10 Mbps connection
Scalability	System shall support up to 10,000 projects without architecture changes	Horizontal scaling via FastAPI workers + PostgreSQL read replicas
Availability	System uptime target	99.5% excluding planned maintenance
Security	All API endpoints require JWT authentication with RBAC	Role-based access: Agency / Analyst / IPMD / Ministry / Public
Security	All data in transit encrypted	TLS 1.3 minimum
Security	All PII (contractor names, officer IDs) pseudonymised in ML pipeline	Pseudonymisation layer in ETL
Explainability	Every risk score must be accompanied by top-5 SHAP driver explanation	Non-negotiable — system must not display bare scores
Auditability	Full audit log of all model predictions, governance actions, and data submissions	Immutable audit log in PostgreSQL with timestamp + user ID
Openness	Zero proprietary dependencies	All libraries: MIT/Apache 2.0/BSD licensed
Portability	Deployable via Docker Compose on any Linux server	Dockerfile + docker-compose.yml provided
Model Governance	No model accuracy claim presented without calibration evidence	Calibration plots (reliability diagrams) published in admin panel

6. Complete System Architecture

6.1 Architecture Overview

PAIMANA-AI follows a modular, layered architecture with clear separation of concerns across five layers: Data Layer, Processing Layer, ML/AI Layer, API Layer, and Presentation Layer. All inter-service communication is via REST or internal Python function calls (no message broker needed at MVP scale).

6.1.1 High-Level Component Diagram (Text Representation)



6.2 Data Layer

6.2.1 Primary Database — PostgreSQL 16

PostgreSQL is the single source of truth for all project data, model metadata, predictions, and audit logs.

Core Tables:

Table	Primary Key	Key Columns	Purpose
projects	project_id (UUID)	sector, ministry, state, sanctioned_cost, approved_date, status	Master project registry
cuf_submissions	submission_id	project_id, reporting_month, revised_cost, expenditure, physical_progress, planned_completion, narrative_text, submitted_by	Monthly CUF data (versioned — never overwrite)
reference_classes	class_id	sector, size_band, region, sample_count, cost_overrun_p50, cost_overrun_p80, cost_overrun_p90, schedule_delay_p50	RCF cluster statistics

predictions	prediction_id	project_id, model_version, prediction_type, predicted_value, confidence_interval_low, confidence_interval_high, shap_values (JSONB), prediction_timestamp	All model outputs
risk_scores	score_id	project_id, reporting_month, cost_risk, schedule_risk, progress_anomaly_score, governance_risk, composite_score, risk_category, data_confidence_score	Composite risk per period
nid_results	nid_id	submission_id, nqc_score, flagged_contradictions (JSONB), llm_model_version	NID analysis results
pbe_cohorts	cohort_id	project_id, cohort_project_ids (ARRAY), ppi_score, ppi_percentile, computed_at	PBE peer assignments
governance_actions	action_id	project_id, action_type, triggered_by, reviewed_by, outcome, timestamp	Audit trail of all governance events
model_registry	model_id	model_type, version, trained_on_date, validation_auc, validation_f1, sector_performance (JSONB), is_active	Model version management

6.3 Processing Layer

6.3.1 ETL Pipeline

- Triggered on each CUF API submission and on the monthly bulk-import schedule
- Steps: Schema validation → Type casting → Null imputation (flagged in DCS) → Derived feature computation → Reference-class assignment → Feature store update
- Technology: Python 3.11, Pandas, SQLAlchemy, Pydantic models for schema validation

6.3.2 Feature Engineering

The following features are computed from raw CUF fields for the ML pipeline:

Feature	Formula / Source	Type	Expected Predictive Role
cost_overrun_ratio	revised_cost / sanctioned_cost	Continuous	Primary overrun signal
schedule_slip_months	(revised_completion - original_completion).months	Continuous	Primary delay signal
expenditure_to_progress_gap	(expenditure/revised_cost) - (physical_progress/100)	Continuous	Anomaly detection
rcf_percentile	Percentile rank in reference class (FR-07)	0–100	De-biased risk baseline
months_since_sanction	(reporting_date - approved_date).months	Integer	Project age / stage signal
progress_velocity_3m	(physical_progress_t - physical_progress_t-3) / 3	Continuous	Pace trend
cost_revision_count	Count of distinct revised_cost values in history	Integer	Instability proxy
agency_track_record	Agency-level historical median overrun (cross-project)	Continuous	Institutional risk

nqc_score	NID output (Section 4.1)	0–100	Misreporting signal
ppi_percentile	PBE output (Section 4.2)	0–100	Peer-relative performance
sector_encoded	One-hot or target-encoded sector label	Categorical	Sector-level risk baseline
reporting_lag_days	Days between period end and submission date	Integer	Data quality / governance signal

6.4 ML / AI Layer

6.4.1 Model Pipeline Architecture

The ML pipeline uses scikit-learn Pipelines to ensure preprocessing and modelling steps are atomic, preventing data leakage. Each pipeline variant is versioned and stored in MinIO.

Step	Implementation	Notes
1. Train/Val/Test Split	Chronological split by project approval date	Prevents temporal leakage (Section 3.3 FR-14)
2. Preprocessing	ColumnTransformer: StandardScaler (numeric), TargetEncoder (categorical)	Fit ONLY on training split
3. Model Training	Baseline → LR → RF → XGBoost → LightGBM	All trained on identical splits for fair comparison
4. Hyperparameter Tuning	Optuna Bayesian optimisation (CV on training set only)	50–100 trials; time-boxed to 2 hours
5. Evaluation	Precision, Recall, F1, AUC-ROC, AUC-PR, Brier score; per-sector breakdown	Recall prioritised for early-warning tasks
6. SHAP Explanation	TreeExplainer for RF/XGB/LGB; LinearExplainer for LR baseline	Stored as JSONB in predictions table
7. Calibration	Platt scaling / isotonic regression on validation set	Required for valid P50/P80/P90 outputs
8. Model Registration	Winner registered in model_registry; previous version retained	Rollback supported

6.4.2 Reference Class Forecasting Engine

- Implementation: SciPy stats distributions (lognormal / gamma fitted to historical overrun ratios per cluster)
- Minimum cluster size: 15 completed projects required before RCF distribution is used; below this, the national-sector distribution is used as fallback with a warning flag
- Output: P50/P80/P90 final cost estimates + probability of >5%, >10%, >20% overrun

6.4.3 NID Implementation Detail

- Ollama server runs as a Docker sidecar on the same host as the API server
- Prompt template is versioned; each NID run records which model and prompt version was used
- LLM output is parsed with Pydantic to extract structured (claim, field, coherence) triples; malformed outputs are flagged as "NID unavailable" rather than failing silently
- NQC score computation is deterministic Python — only the claim extraction uses the LLM

6.5 API Layer — FastAPI

Endpoint	Method	Description	Auth
/api/v1/projects	GET	List projects with filters (sector, state, risk level, ministry)	IPMD / Ministry / Agency (own)

/api/v1/projects/{id}/risk	GET	Composite risk score + SHAP explanation for a project	IPMD / Agency (own)
/api/v1/projects/{id}/rcf	GET	Reference-class benchmarking output + P50/P80/P90 estimates	IPMD / Agency (own)
/api/v1/projects/{id}/nid	GET	NID result: NQC score + flagged contradictions	IPMD only
/api/v1/projects/{id}/pbe	GET	Peer cohort + PPI percentile + league table	IPMD / Agency (own — anonymised peers)
/api/v1/submissions	POST	Submit new CUF data for a project	Agency (own)
/api/v1/dashboard/national	GET	Aggregated national risk summary (sector × state heatmap)	Ministry / IPMD
/api/v1/governance/queue	GET	Projects pending IPMD review, sorted by risk score	IPMD only
/api/v1/governance/action	POST	Record governance action on a flagged project	IPMD only
/api/v1/admin/models	GET	Model registry: all versions, performance metrics	Analyst (DIID)
/api/v1/public/summary	GET	Aggregated sector/state risk summary (read-only, no project IDs)	Public

6.6 Presentation Layer — React Dashboard

6.6.1 Key Views

View	User	Key Components
National Risk Map	Ministry / IPMD	Choropleth map of India coloured by state-level risk score; sector-level bar chart; top-10 highest-risk projects table
Project Detail	IPMD / Agency	Risk gauge (composite + components); P50/P80/P90 cost/time forecast bars; SHAP waterfall chart; NID contradiction cards; PBE peer scatter plot; monthly trend lines; governance action panel
Agency Dashboard	Agency	Own project list with risk badges; NQC trend chart; PBE percentile vs. cohort; CUF submission status and DCS
Governance Queue	IPMD	Sorted queue of flagged projects; one-click review initiation; action recording form; SLA timer per project
Model Performance	Analyst (DIID)	Per-model AUC/F1/Brier score table; per-sector performance breakdown; calibration plot; feature importance chart; temporal drift monitoring
Public Summary	Public	Aggregated sector-level overrun statistics; trend charts; no project-level or agency-level data

6.7 Infrastructure & Deployment

Component	Technology	Spec (Minimum)
Application Server	FastAPI + Uvicorn + Gunicorn	4 vCPU, 16 GB RAM (scales horizontally)
ML Compute	Python workers (APScheduler)	Same server; GPU not required at MVP scale
LLM Runtime (NID)	Ollama with Mistral-7B-Instruct	8 GB RAM additional; CPU inference for 2000 projects/month is feasible
Database	PostgreSQL 16	4 vCPU, 32 GB RAM, 500 GB SSD

Cache	Redis 7	2 GB RAM; caches risk scores and dashboard aggregations
Object Storage	MinIO (S3-compatible)	For ML model artefacts and OCMS bulk imports; 100 GB
Frontend	React 18 + Nginx	Served as static assets; Nginx reverse proxy
Containerisation	Docker + Docker Compose	Single docker-compose.yml for one-command deployment
CI/CD	GitHub Actions	Automated tests (pytest) + linting on every PR
Monitoring	Prometheus + Grafana	API latency, model drift metrics, DB health

7. Complete Open-Source Technology Stack

Layer	Library/Tool	Version	Purpose	License
Backend	Python	3.11	Core language	PSF
Backend	FastAPI	0.111	REST API framework	MIT
Backend	Pydantic	2.7	Data validation & schemas	MIT
Backend	SQLAlchemy	2.0	ORM and query builder	MIT
Backend	Alembic	1.13	Database migrations	MIT
Backend	APScheduler	3.10	Scheduled ML re-training	MIT
ML	scikit-learn	1.5	Pipeline, preprocessing, RF baseline	BSD-3
ML	XGBoost	2.0	Gradient boosting classifier/regressor	Apache-2.0
ML	LightGBM	4.3	Fast gradient boosting	MIT
ML	SHAP	0.45	Model explainability (TreeExplainer)	MIT
ML	Optuna	3.6	Hyperparameter optimisation	MIT
ML	SciPy	1.13	Statistical distributions for RCF	BSD-3
ML	imbalanced-learn	0.12	Class imbalance handling (SMOTE)	MIT
NID	Ollama	0.1.x	Local LLM inference server	MIT
NID	LLaMA-3-8B-Instruct	Meta	Narrative analysis LLM (local)	Meta Llama 3 Community
PBE	scikit-learn NearestNeighbors	1.5	k-NN cohort identification	BSD-3
Database	PostgreSQL	16	Primary data store	PostgreSQL License
Cache	Redis	7	Score caching & session store	BSD-3
Storage	MinIO	Latest	Model artefact store	AGPL-3.0
Frontend	React	18	SPA framework	MIT
Frontend	Recharts	2.12	Data visualisation charts	MIT
Frontend	TailwindCSS	3.4	Utility-first CSS	MIT
Frontend	Mapbox-gl (OSM tiles)	3.3	Choropleth India map	BSD-3
Frontend	React Query	5.x	API state management	MIT
DevOps	Docker	26	Containerisation	Apache-2.0
DevOps	Nginx	1.26	Reverse proxy + static file serving	BSD-2
DevOps	Prometheus + Grafana	Latest	Monitoring & alerting	Apache-2.0
Testing	pytest + httpx	Latest	Backend unit & integration tests	MIT
Testing	Playwright	1.44	Frontend E2E tests	Apache-2.0

8. Build Feasibility Analysis — What Is Completely Doable

8.1 Executive Feasibility Summary

Honest Assessment

Based on the global comparative analysis and the SIH problem statement, the following breakdown honestly assesses what a skilled team of 4–6 developers can deliver:

- ✓ FULLY BUILDABLE in SIH timeframe (36–48 hours): ~65% of total system
- ⚠ PARTIALLY BUILDABLE (demo-ready, not production-hardened): ~20%
- ✗ OUT OF SCOPE for hackathon (Phase 2/3 items): ~15%

8.2 Detailed Feasibility Matrix

Feature / Component	Feasibility	Effort (hrs)	Blocker / Risk	Evidence Grade
PostgreSQL schema + migrations	✓ FULLY BUILDABLE	4–6	None	—
FastAPI REST API skeleton + JWT auth	✓ FULLY BUILDABLE	6–8	None	—
CUF submission + validation endpoint	✓ FULLY BUILDABLE	4–6	Need sample CUF schema	—
Data Confidence Score (DCS)	✓ FULLY BUILDABLE	4–6	None	C/D
OCMS historical data import (CSV)	✓ FULLY BUILDABLE	3–5	Depends on OCMS data availability	—
Reference Class Benchmarking (RCF)	✓ FULLY BUILDABLE	8–12	Need completed project labels	A/B
Random Forest baseline model	✓ FULLY BUILDABLE	4–6	Need ground-truth overrun labels	B
XGBoost / LightGBM models	✓ FULLY BUILDABLE	4–6	Same as above	B
SHAP explainability layer	✓ FULLY BUILDABLE	3–4	None (add-on to tree models)	B
Anomaly detection (rule-based)	✓ FULLY BUILDABLE	4–6	Threshold calibration needed	B/D
Composite risk score computation	✓ FULLY BUILDABLE	4–6	Weight calibration needed	E
P50/P80/P90 probabilistic forecasts	✓ FULLY BUILDABLE	4–6	From RCF distributions	A
Governance escalation queue	✓ FULLY BUILDABLE	4–6	None	A
React dashboard — project detail view	✓ FULLY BUILDABLE	8–12	None	—
React dashboard — national risk map	✓ FULLY BUILDABLE	6–8	OSM/Mapbox tile integration	—
NID — LLM claim extraction (Ollama)	✓ FULLY BUILDABLE	8–12	Needs 8GB RAM for LLM; prompt engineering	E
NID — cross-check rule engine	✓ FULLY BUILDABLE	4–6	None	E

NID — NQC score + dashboard card	✓FULLY BUILDABLE	3–4	None	E
PBE — cosine similarity cohort (k-NN)	✓FULLY BUILDABLE	6–8	None	E
PBE — PPI score computation	✓FULLY BUILDABLE	4–6	Stage-normalisation needs calibration	E
PBE — peer league table (anonymised)	✓FULLY BUILDABLE	4–6	None	E
Model benchmarking comparison table	✓FULLY BUILDABLE	3–4	None — report all models fairly	B
Time-aware train/test split	✓FULLY BUILDABLE	2–3	None	B
Docker Compose deployment	✓FULLY BUILDABLE	3–4	None	—
EVM linear regression baseline	✓FULLY BUILDABLE	2–3	None	B
Per-sector model performance report	✓FULLY BUILDABLE	2–3	Needs sufficient per-sector samples	B
Contractor performance tracking	⚠ PARTIAL	4–6	Contractor ID not confirmed in CUF	B
Land acquisition status feature	⚠ PARTIAL	2–3	Field availability not confirmed	B
Agency portfolio intelligence (PBE)	⚠ PARTIAL	4–6	Needs multiple projects per agency in data	E
Monte Carlo risk-register simulation	⚠ PARTIAL (stub)	6–8	Full risk register not in CUF	A
Monthly automated re-training	⚠ PARTIAL (stub)	4–6	Needs APScheduler + prod deployment	—
BIM / GIS integration	✗ OUT OF SCOPE	N/A	Hardware/tooling dependency	A/B
IoT sensor fusion	✗ OUT OF SCOPE	N/A	Hardware dependency	B/D
Drone / CV progress verification	✗ OUT OF SCOPE	N/A	Field infrastructure needed	D
Digital twin construction	✗ OUT OF SCOPE	N/A	Multi-year undertaking	B/D
Post-completion asset monitoring	✗ OUT OF SCOPE (schema stub only)	2	Schema design only	B/D

8.3 Critical Path for SIH Build

The following sequence ensures the highest-value features are working first:

Hour Range	Milestone	Owner Role
0–4	Database schema, Docker Compose, Alembic migrations running	Backend Lead
4–8	FastAPI skeleton, JWT auth, CUF submission endpoint live	Backend Lead
8–12	OCMS historical data imported; Reference Class Benchmarking computing distributions	Data/ML Lead
12–18	Random Forest + XGBoost + LightGBM trained; SHAP integrated; baseline comparison table	ML Lead

18–22	Composite risk score, P50/P80/P90 outputs, DCS — all via API	Backend + ML
22–28	React dashboard: national map + project detail view (risk, SHAP waterfall, P50/P80/P90)	Frontend Lead
28–34	NID: Ollama running, prompt engineering, NQC score, contradiction cards in dashboard	ML + Frontend
34–38	PBE: cohort identification, PPI computation, peer league table in dashboard	ML + Frontend
38–42	Governance queue, audit trail, escalation workflow, agency dashboard	Backend + Frontend
42–48	Testing, bug fixes, Docker deployment, README, demo recording	All

9. Data Strategy & Model Validation

9.1 Data Audit Priorities (Before Any Modelling)

Per the Blueprint's Section 6, the following fields must be confirmed available before model training begins. Supervised overrun prediction is impossible without actual completion labels.

Field	Priority	Action if Missing
Actual final cost (completed projects)	CRITICAL — P0	Without this, supervised training not possible. Use RCF-only approach until available.
Actual completion date (completed projects)	CRITICAL — P0	Same as above for schedule models.
Contractor ID / contractor information	HIGH — P1	Approximate via agency + sector clustering. Flag as missing feature.
Land acquisition status	HIGH — P1	Use scope-change proxy (sudden revised_cost jump > 15%). Flag limitation.
Scope change / variation order records	HIGH — P1	Proxy: count of distinct revised_cost values. Flag limitation.
Location (state / district)	MEDIUM — P1	Often derivable from implementing agency. Enrich programmatically.

9.2 Time-Aware Validation Protocol

All models must be trained and evaluated using chronological splits. The exact split dates will be determined by the actual OCMS/PAIMANA data extent discovered in the audit. The following structure applies:

- Training set: earliest 60% of completed projects by approval date
- Validation set: next 20% by approval date (hyperparameter tuning and threshold calibration)
- Test set: most recent 20% of completed projects (final reported performance)
- Rolling forward validation: for the early-warning task, re-evaluate at each N-month-before-completion checkpoint

9.3 Model Performance Reporting Standard

The SIH submission must include a performance comparison table with the following structure for every prediction task (cost classification, schedule classification, early-warning):

Model	AUC-ROC	F1 (0.5 threshold)	Recall	Precision	Brier Score	Notes
Naive baseline (sector avg)	TBD	TBD	TBD	TBD	TBD	Lower bound
Logistic Regression	TBD	TBD	TBD	TBD	TBD	Statistical baseline
EVM / RCF statistical	TBD	TBD	TBD	TBD	TBD	International practice baseline
Random Forest	TBD	TBD	TBD	TBD	TBD	Primary ensemble
XGBoost	TBD	TBD	TBD	TBD	TBD	Primary ensemble
LightGBM	TBD	TBD	TBD	TBD	TBD	Primary ensemble
Best ensemble (selected)	TBD	TBD	TBD	TBD	TBD	Production model

All "TBD" values will be filled with empirically measured results from the actual PAIMANA/OCMS dataset. Presenting pre-filled numbers before training is explicitly prohibited in this document.

10. Governance & Human-in-the-Loop Design

10.1 Core Governance Principle

Non-Negotiable Safety Constraint

PAIMANA-AI shall NEVER autonomously approve, reject, or reallocate project funding.
The system's role is detection, explanation, and routing only.
Every consequential decision rests with an authorised human reviewer.
This is a deliberate design choice, not a current technical limitation.

10.2 Escalation Workflow

Risk Category	Automated Action	Required Human Action	SLA
LOW	No alert; record in dashboard	Optional review on next reporting cycle	—
MODERATE	Flag in agency's own dashboard; record	Agency to explain deviation in next CUF narrative	5 business days
HIGH	Auto-route to IPMD review queue; email notification to IPMD officer	IPMD officer must initiate structured review	10 business days
VERY HIGH	Auto-route to IPMD queue; flag to Ministry dashboard; block gate advance	IPMD structured review + Ministry notification required	5 business days
CRITICAL	All of the above + NID contradiction flag attached to case file	IPMD priority review; independent verification recommended	2 business days

10.3 International Governance Principles Applied

- South Korea (PFS): the reviewing IPMD officer must be institutionally separate from the implementing agency — no self-review
- Netherlands (MIRT): governance gate blocks are automatic once risk score crosses VERY HIGH — not advisory
- United States (PMOC): escalation is rule-based, not left to discretionary reporting
- Australia (Confidence Index): the risk score and DCS are published to authorised dashboard users — transparency is the accountability mechanism

11. Failure Modes & Mitigations

Failure Mode	Impact	Mitigation Built Into System
Ground-truth labels missing (no completed project records)	Supervised models cannot be trained	Fall back to RCF-only mode; clearly flag to users that ML models are unavailable; do not display false accuracy
Data leakage (random instead of chronological split)	Inflated reported accuracy that fails in production	Time-aware split enforced in code; unit test verifies no future data in training set
Concept drift (procurement rules / economic conditions change)	Model degrades over time silently	Monthly retraining + Prometheus alert if validation AUC drops >5% month-on-month
Sector imbalance (rare project types have tiny samples)	Good pooled accuracy hides poor per-sector performance	Per-sector performance reported separately; sectors with < 30 samples excluded from sector-specific models
Strategic misreporting (agencies game the numbers)	ML model trained on manipulated data learns wrong patterns	NID detects narrative-quantitative contradictions; anomaly module flags expenditure-progress gaps; both feed as features into ML model
LLM (NID) producing inconsistent outputs	NID results are unreliable or hallucinated	NQC score is computed by deterministic Python; LLM only performs claim extraction; malformed outputs flagged as "NID unavailable", not silently discarded
False positive overload (too many HIGH flags)	IPMD review queue overwhelmed; staff start ignoring flags	Recall vs. precision trade-off explicitly calibrated to keep false positive rate < 15%; queue SLA monitoring in Grafana
False negatives (missed high-risk projects)	Intervention opportunity lost	False negatives are the higher-cost error; threshold tuning explicitly favours recall; residual risk acknowledged to users
Model presented as certain when uncertain	Decision-makers over-trust point estimates	All outputs include confidence intervals and DCS; Brier score (calibration) published in admin panel
PBE cohort too small for meaningful comparison	PPI scores unreliable for rare project types	Minimum cohort size = 5 projects; below this, PBE shows "Insufficient peer data" rather than a score

12. Appendix — Global Practice to PAIMANA Feature Mapping

Summary of which international innovations each PAIMANA-AI feature is grounded in:

PAIMANA-AI Feature	Grounded In	Evidence Grade
Reference Class Benchmarking	UK RCF mandate (IPA/DfT, 2003); Park 2021 before/after analysis	A/B
P50/P80/P90 probabilistic forecasts	UK RCF uplift logic + US FTA Monte Carlo (GAO-23-105479)	A
Ensemble ML (RF/XGB/LGB)	South Korea bidding-stage studies (Moon et al. 2020); cross-cutting literature review (196 + 27 papers)	B
SHAP explainability	Implicit requirement of all governance-linked systems reviewed; consistent with Netherlands, Korea, US, Australia models	B/E
Data Confidence Score	Singapore CORENET X data-quality-at-source principle (BCA, Oct 2025)	C/D
Anomaly detection	China Digital Twin Railway monitoring concept, adapted for structured tabular data	B/D
Composite risk score + Confidence Index	Australia Infrastructure Priority List + Confidence Index (Infrastructure Australia, 2023–24 Halton Review)	A/D
Governance escalation gates	Netherlands MIRT 4-phase gates; South Korea PFS; US PMOC/GAO escalation (GAO-23-105479)	A
Human-in-the-loop workflow	Universal principle across all 10 reviewed systems — no system reviewed automates funding decisions	A
NID (Narrative Inconsistency Detector)	NOVEL — no direct international precedent. Addresses misreporting gap identified in Japan, China, Gulf country profiles	E (novel)
PBE (Peer-Pressure Benchmarking Engine)	NOVEL — extends Australia's Confidence Index concept and South Korea's PFS accountability principle into a continuous, dynamic mechanism	E (novel, grounded in A/B evidence of peer-accountability effects)
Schema extensibility for post-completion monitoring	China Digital Twin Railway; NEOM cognitive digital twin — schema designed for extension without platform rebuild	B/D

13. Document Versioning & Disclaimer

This SRS is prepared for SIH 2026 submission purposes. All proposed ML thresholds, risk category boundaries, feature weights, and component score formulas are presented as proposed architecture pending calibration on actual PAIMANA/OCMS data. No numeric accuracy claim for PAIMANA is presented as validated — all will require empirical testing on the real dataset as the first task of the build. The international accuracy figures cited (e.g. 61% Korean bidding-stage study, 85–90% large-dataset deep learning) are from distinct academic studies and are indicative of relative model rankings, not expected PAIMANA performance.

References: Full reference list is documented in the accompanying SIH2026_PAIMANA_Global_Comparative_Analysis.pdf and SIH2026_PAIMANA_Global_Blueprint.pdf. All academic sources are peer-reviewed (Grade B or higher per the Blueprint's evidence hierarchy). All government/institutional sources are official publications (Grade A).

ADDENDUM

Repository Structure & Two Additional Innovative Features

14. Repository & Codebase File Structure

PAIMANA-AI is organised as a single monorepo so that the data pipeline, model training code, API, and dashboard stay version-locked together — critical for a government system where a model version and the API contract that serves it must never drift apart silently. The layout below maps directly onto the five architecture layers in Section 6 and onto Features 1–4 (NID, PBE, and the two features introduced in this addendum).

14.1 Top-Level Layout

```
paimana-ai/
├── backend/                                     # FastAPI application (API + ML/AI + Processing layers)
│   ├── app/
│   │   ├── main.py                             # FastAPI entrypoint, router registration, middleware
│   │   ├── core/                               # config.py, security.py (JWT), logging.py
│   │   └── api/
│   │       ├── v1/
│   │       │   ├── projects.py                 # /projects, /projects/{id}/risk, /rcf
│   │       │   ├── submissions.py              # POST /submissions (CUF ingestion)
│   │       │   ├── governance.py               # /governance/queue, /governance/action
│   │       │   ├── network.py                  # NEW – Feature 3 (contagion graph) endpoints
│   │       │   ├── simulate.py                 # NEW – Feature 4 (what-if simulator) endpoints
│   │       │   ├── nid.py                      # Feature 1 (Narrative Inconsistency Detector)
│   │       │   ├── pbe.py                      # Feature 2 (Peer Benchmarking Engine)
│   │       │   └── admin.py                    # /admin/models, model registry
│   │       ├── models/                         # SQLAlchemy ORM models (1 file per table, Section 6.2)
│   │       ├── schemas/                       # Pydantic request/response schemas
│   │       └── services/                       # business logic, decoupled from FastAPI routes
│   │           ├── rcf_engine.py
│   │           ├── risk_scoring.py
│   │           ├── data_confidence.py
│   │           ├── anomaly_detection.py
│   │           ├── nid_service.py               # Ollama client + claim-extraction prompts
│   │           ├── pbe_service.py               # cohort kNN + PPI computation
│   │           ├── contagion_graph.py           # NEW – Feature 3 graph build + diffusion
│   │           └── counterfactual_sim.py        # NEW – Feature 4 scenario re-scoring
│   ├── db/
│   │   ├── session.py
│   │   └── migrations/                         # Alembic versioned migrations
│   ├── tests/                                 # pytest – unit + API integration tests
│   └── requirements.txt
├── ml_pipeline/                               # Offline training – NOT imported by the live API
│   ├── data_prep/                             # ETL, feature engineering scripts (Section 6.3)
│   ├── training/
│   │   ├── train_baseline.py                   # naive + linear regression
│   │   ├── train_rcf.py                       # reference-class distribution fitting
│   │   ├── train_ensemble.py                   # RF / XGBoost / LightGBM + Optuna tuning
│   │   ├── train_contagion_weights.py          # NEW – edge-weight calibration for Feature 3
│   │   └── evaluate.py                         # Section 8 benchmarking harness
│   ├── notebooks/                             # exploratory analysis (not shipped to prod)
│   └── artifacts/                             # model binaries staged for MinIO upload (gitignored)
├── frontend/                                  # React 18 SPA (Presentation layer)
│   ├── src/
│   │   ├── pages/
│   │   │   ├── NationalRiskMap.jsx
│   │   │   ├── ProjectDetail.jsx
│   │   │   ├── AgencyDashboard.jsx
│   │   │   ├── GovernanceQueue.jsx
│   │   │   ├── NetworkExplorer.jsx             # NEW – Feature 3 force-directed graph view
│   │   │   ├── DecisionCockpit.jsx             # NEW – Feature 4 what-if simulator view
│   │   │   └── PublicSummary.jsx
│   │   ├── components/
│   │   └── charts/                             # RiskGauge, SHAPWaterfall, PPIScatter, ForecastBands
```

graph/	# NEW – BlastRadiusGraph.jsx, EdgeLegend.jsx
simulator/	# NEW – InterventionSliders.jsx, ScenarioCard.jsx,
	# CostOfInactionClock.jsx
common/	# Nav, Auth guard, Table, Badge
services/	# api.js (Axios client), auth.js, websocket.js
state/	# React Query hooks / Zustand store
package.json	
infra/	# Deployment (Section 6.7)
docker-compose.yml	# api + db + redis + minio + ollama + nginx + frontend
Dockerfile.backend	
Dockerfile.frontend	
nginx/	
monitoring/	# prometheus.yml, grafana dashboards (JSON)
data/	# sample/seed data only – real CUF data never committed
seed/	
schema/	# data dictionary CSV/JSON (Section 6, 15)
docs/	# this SRS, model cards, ADRs, diagrams
SIH2026_PAIMANA_SRS_Architecture.docx	
model_cards/	
adr/	# architecture decision records
scripts/	# monthly_refresh.sh, seed_db.py, backfill_ocms.py
.github/workflows/	# ci.yml (pytest + lint), cd.yml (build & push images)
.env.example	
README.md	

14.2 Placement Rules — Why Files Live Where They Do

- backend/app/services/ holds all business logic as plain Python classes/functions with no FastAPI decorators, so every model (RCF, NID, PBE, contagion graph, simulator) can be unit-tested and reused by the offline ml_pipeline/ without importing the web framework.
- ml_pipeline/ is intentionally separate from backend/ — training is a batch job that produces a versioned artifact (Section 6.4.1 step 8); the API only ever loads a frozen model file from MinIO, so a bad training run can never affect the live service until an artifact is explicitly promoted.
- Every NEW component introduced by Features 3 and 4 (contagion_graph.py, counterfactual_sim.py, train_contagion_weights.py, NetworkExplorer.jsx, DecisionCockpit.jsx) slots into the existing folders rather than requiring a new top-level module — confirming the features are additive, not architectural rewrites.
- frontend/src/components is split by concern (charts / graph / simulator / common) so the two new views can be built and demoed independently by different team members without merge conflicts — directly supporting the 6-person role split in the Approach document.
- docs/adr/ (architecture decision records) is a lightweight addition recommended for the SIH judging round: a one-page note per major design choice (e.g. ‘why rule-based diffusion, not a GNN, for Feature 3 MVP’) demonstrates engineering maturity to technical judges.

15. Two Additional Innovative Features (Full-Stack)

Why These Two, and Why Now

Features 1 (NID) and 2 (PBE) in Section 4 fix data-trust and accountability problems using data PAIMANA already has, project-by-project. The two features below extend the platform in a direction none of the ten countries reviewed have built: treating the PAIMANA portfolio as a connected SYSTEM (Feature 3) and turning the dashboard from a diagnostic tool into a decision-support tool that can quantify the value of an intervention before it is made (Feature 4). Both ship a real frontend surface and a real backend service — neither is a slide-only concept.

15.1 — INNOVATIVE FEATURE 3: Systemic Risk Contagion Network ("Blast-Radius Engine")

★NOVEL ★MVP-FEASIBLE ★No country reviewed models cross-project contagion

15.1.1 Problem It Solves

Every model in Sections 6–10 — RCF, the ensemble classifiers, NID, PBE — scores ONE project at a time using that project's own history. But real overruns are frequently correlated across projects through shared resources: the same

contractor stretched across 12 sites, the same district land acquisition officer bottlenecked on 5 projects at once, the same steel/cement supplier hit by a price shock affecting every project that sources from it. A project-independent model cannot see this until each affected project's OWN CUF data eventually deteriorates — by which point the early-warning value is largely gone. No system in the ten-country comparative analysis (Section 3, comparative document) models risk propagation between projects; each treats appraisal or monitoring as a single-project exercise.

15.1.2 How It Works

- Build a heterogeneous graph: nodes = {projects, contractors, implementing agencies, districts, material-supplier categories}; edges = declared relationships from CUF fields (contractor ID, agency, project location) plus a supplier-category edge derived from project sector.
- Weight each edge type by its historically observed 'risk transferability': e.g. shared-contractor edges are calibrated against how often two projects sharing a contractor show cost/schedule risk within 2 months of each other in historical OCMS data (train_contagion_weights.py in Section 14.1).
- When any project's composite risk score moves by more than a threshold in one reporting cycle, run a bounded diffusion pass (1–2 hops only, damped like a simplified PageRank) that adds a small 'propagated risk' component to its graph-neighbours' composite score — clearly labelled as propagated, never merged invisibly into the neighbour's own score.
- Surface a dedicated 'second-order early warning' alert class: projects that show NO deterioration in their own CUF data yet, but are within 1 hop of a newly-flagged high-risk project through a shared contractor or agency.
- Stretch goal (Phase 2, not MVP): replace the rule-based diffusion weights with a learned Graph Neural Network (PyTorch Geometric, GraphSAGE) once enough propagation events have been observed to train on — explicitly deferred per the same data-volume discipline used for deep learning in Section 8.

15.1.3 Backend Implementation

Component	Technology	Detail
Graph store	NetworkX (MVP) → optional Neo4j (Phase 2)	In-memory MultiDiGraph rebuilt on each ETL cycle from projects, contractors, agencies, districts tables; NetworkX is sufficient at ~2,000 project-scale, avoiding a new DB dependency for MVP
Edge weight calibration	Python + scikit-learn (train_contagion_weights.py)	Historical co-occurrence of risk events within N months, per edge type, converted to a diffusion coefficient (0–1) per edge type
Diffusion algorithm	Custom Python, NetworkX	Damped 1–2-hop weighted risk spreading (analogous to personalised PageRank), re-run every monthly re-scoring cycle (Section 6.4.1)
New tables	PostgreSQL	entities(entity_id, type, name), project_relationships(project_id, entity_id, relationship_type, weight), contagion_alerts(project_id, source_project_id, propagated_risk_delta, hop_distance, computed_at)
New API endpoints	FastAPI (network.py)	GET /api/v1/projects/{id}/network (ego-subgraph, 2 hops); GET /api/v1/network/contagion-alerts (portfolio-wide second-order alerts, IPMD only)

15.1.4 Frontend Implementation

Component	Technology	Detail
Network Explorer page	React + react-force-graph (WebGL force-directed layout)	New page (frontend/src/pages/NetworkExplorer.jsx); nodes coloured by risk category, sized by project value; edge thickness = relationship weight
Blast-radius interaction	D3 force simulation + React state	Clicking any contractor/agency/district node animates and highlights every connected project, with a live-updating propagated-risk badge on each
Edge-type filter	Toggle chips (React)	Show/hide contractor / agency / district / supplier-category edges independently, so a reviewer can isolate 'which of my projects share this contractor'
Second-order alert banner	Reusable Alert component on Project Detail page	"This project's own data looks stable, but 3 sister-projects of its contractor were flagged this month — propagated risk +<value>", with a link into the Network Explorer pre-filtered to that contractor
Governance queue integration	Extend existing GovernanceQueue.jsx	Second-order alerts appear in the same IPMD review queue as first-order alerts, tagged 'Propagated' so

Worked Example — Blast-Radius Alert

Contractor: Meridian Infra Pvt Ltd | Active PAIMANA projects: 9

Event: Project NH-27 Widening (Karnataka) crosses HIGH risk this month — schedule slip driven by contractor equipment shortage (per NID-extracted narrative flag).

Blast-Radius Engine propagates: 4 other Meridian-contracted projects (Telangana bridge, Rajasthan highway, 2 Maharashtra flyovers) each receive a +12 to +19 point propagated-risk addition, despite their own physical/financial progress fields showing no deterioration this month.

Result: all 5 projects appear together in the IPMD Governance Queue, tagged so reviewers immediately see this is a CONTRACTOR-CAPACITY issue, not five unrelated project problems — changing the intervention from five separate reviews to one conversation with Meridian Infra.

15.1.5 Why This Is Genuinely Extraordinary

Every mechanism in the global comparative analysis (RCF, PFS, MIRT, Monte Carlo, digital twins) evaluates projects one at a time. The Blast-Radius Engine is the first mechanism in this entire document set to treat PAIMANA's ~1,981 projects as a connected system rather than as 1,981 independent records — turning a shared-resource weakness (contractor overextension, agency capacity limits) from an invisible cross-cutting risk into a named, visualised, and actionable one, months before it would show up in any single project's own numbers.

15.2 — INNOVATIVE FEATURE 4: Counterfactual "What-If" Decision Simulator ("Decision Cockpit")

★NOVEL ★MVP-FEASIBLE (fast path) ★No country reviewed quantifies intervention value pre-decision

15.2.1 Problem It Solves

Sections 7–10 and Feature 1/2 all produce diagnoses: a risk score, a narrative-contradiction flag, a peer-percentile. None of them answer the question an IPMD reviewer or Ministry official actually needs answered under time pressure: 'of the three things I could do about this project right now, which one moves the needle the most, and what does waiting one more month cost?' The US GAO/PMOC model (Section 3.8) and Netherlands MIRT gates (Section 3.2) both wire predictions to a review decision, but neither system reviewed quantifies the expected impact of a SPECIFIC intervention before it is taken. That gap is what the Decision Cockpit closes.

15.2.2 How It Works

- Define a small, explicit set of intervention levers that map to real governance actions already available to IPMD/Ministry users: release-funds-early, replace-contractor, expedite-land-acquisition, freeze-scope, escalate-to-independent-review.
- Each lever is pre-mapped (via a simple, transparent precedence rule-set, not a hidden model) to the specific CUF-derived features it would plausibly change — e.g. expedite-land-acquisition affects `schedule_slip_months` and `progress_velocity_3m`, but is NOT allowed to directly alter `sanctioned_cost` — preventing the tool from implying a causal link the data cannot support.
- When a reviewer selects a lever (or combination), the perturbed feature vector is re-scored through the SAME trained ensemble model and SHAP explainer used for live predictions (Section 6.4.1), producing a new composite risk score, a new P50/P80/P90 forecast band (Section 14 in the Blueprint), and a SHAP-delta waterfall showing exactly which features moved and by how much.
- A two-tier compute design keeps this interactive: a FAST PATH uses precomputed marginal-effect curves (from batch scoring) for real-time slider feedback in the UI, and an EXACT PATH re-runs full model + SHAP inference only when the reviewer commits to a specific scenario — balancing UI responsiveness against computational cost.
- A 'Cost of Inaction' clock combines the project's current risk trend with its reference-class distribution (Section 5, Lesson 1) to express the marginal cost of delaying a decision in concrete, monthly terms (₹ crore/month of expected additional overrun, or days/month of expected additional slip) — not just an abstract risk-score number.
- Every committed scenario is written to `governance_actions` (already in the schema, Section 6.2.1) together with its PREDICTED counterfactual effect, so months later the actual outcome can be compared against the prediction — directly extending the feedback-loop mechanism recommended in the Blueprint (Section 5, Lesson 13) from 'did our forecast come true' to 'did our recommended action work.'

15.2.3 Backend Implementation

Component	Technology	Detail
Lever → feature mapping	Declarative YAML/JSON config, loaded at startup	Human-readable, auditable mapping of each intervention lever to the features it may perturb and the direction of plausible effect — reviewable by a non-engineer (e.g. IPMD domain lead) before deployment
Fast-path marginal effects	Precomputed during batch scoring (APScheduler job)	For each active project, store the model's local sensitivity (partial-dependence-style) to each lever-affected feature, cached in Redis for sub-second slider feedback
Exact-path re-inference	Same scikit-learn Pipeline + SHAP TreeExplainer used in production (Section 6.4.1)	Triggered only on 'Run Full Simulation' or 'Commit Scenario'; guarantees the number a reviewer acts on is never an approximation
Cost-of-inaction calculator	Python service combining risk-trend slope + RCF distribution (Section 6.4.2)	Outputs ₹/month and days/month figures with an explicit 'correlational estimate, not a guarantee' caveat surfaced in every response
New tables	PostgreSQL	simulation_scenarios(scenario_id, project_id, levers_applied JSONB, predicted_risk_delta, predicted_cost_of_inaction, created_by, created_at); extend governance_actions with a nullable scenario_id FK
New API endpoints	FastAPI (simulate.py)	GET /api/v1/projects/{id}/simulate/fast?levers=... (cached marginal-effect lookup); POST /api/v1/projects/{id}/simulate/exact (full re-inference); POST /api/v1/projects/{id}/simulate/commit (logs chosen scenario to governance_actions)

15.2.4 Frontend Implementation

Component	Technology	Detail
Decision Cockpit page	React (frontend/src/pages/DecisionCockpit.jsx)	New page reachable from the Project Detail view; split-panel layout — levers on the left, live-updating forecast on the right
Intervention sliders/toggles	React components (components/simulator/InterventionSliders.jsx)	One control per lever; dragging a slider calls the FAST-PATH endpoint (debounced) and animates the risk gauge and P50/P80/P90 bars in real time
Scenario comparison cards	components/simulator/ScenarioCard.jsx + Recharts	Side-by-side cards for 'Do nothing' vs. each selected combination of levers, each showing predicted risk delta, SHAP-style plain-language rationale, and cost-of-inaction saved
Cost-of-Inaction clock	components/simulator/CostOfInactionClock.jsx	A live countdown-style widget showing ₹/day or days/day of expected additional overrun accruing while the project remains un-actioned — designed to visually motivate timely governance decisions without being alarmist
Commit & audit trail	Extends GovernanceQueue.jsx	'Commit to Action' button posts to /simulate/commit and immediately shows the logged action in the governance audit trail, with the predicted effect attached for later comparison against the actual outcome

Worked Example — Decision Cockpit Session

Project: Coastal Highway Package 3 | Current composite risk: 71 (HIGH) | Data confidence: 88%

Reviewer drags 'Expedite land acquisition' to '+2 months faster':

→ Fast-path forecast updates instantly: risk 71 → 54, schedule P80 improves by ~5.5 months

Reviewer adds 'Replace contractor' (tier-1 substitute) on top:

→ Fast-path forecast: risk 54 → 46, but cost P50 rises slightly (remobilisation cost modelled)

Cost-of-Inaction clock shows: ₹1.8 Cr/month and 0.6 months/month of expected additional slip while the project remains un-actioned at its current risk level.

Reviewer clicks 'Run Full Simulation' on the combined scenario → exact re-inference confirms the

fast-path estimate within tolerance → clicks ‘Commit to Action’ → logged to governance_actions with the predicted risk delta attached for outcome comparison in 6 months.

15.2.5 Why This Is Genuinely Extraordinary

Reference Class Forecasting, Monte Carlo risk registers, and Confidence Indices (Sections 3.1, 3.8, 3.9 of the Blueprint) all answer ‘how risky is this project.’ None of the ten systems reviewed answer ‘what is this specific decision worth.’ The Decision Cockpit is the only mechanism in this document set that converts a prediction into a decision-support number — and, by logging every committed scenario against its predicted effect, it is also the only feature that creates a ready-made dataset for validating whether PAIMANA’s own recommendations actually work, closing the loop that the US GAO ‘Before and After’ studies (Section 3.8) show is essential but that requires years to build manually.

15.3 Summary — All Four Innovative Features at a Glance

#	Feature	Layer It Extends	Core Novelty
1	Narrative Inconsistency Detector (NID)	ML/AI Layer (Section 4.1)	Cross-checks free-text claims against numeric CUF fields using a local LLM
2	Peer-Pressure Benchmarking Engine (PBE)	ML/AI Layer (Section 4.2)	Continuous, dynamic peer-cohort accountability, not just at appraisal
3	Systemic Risk Contagion Network	ML/AI + Presentation Layer (Section 15.1)	Treats the portfolio as a connected graph; propagates risk across shared contractors/agencies before a project’s own data deteriorates
4	Counterfactual What-If Decision Simulator	ML/AI + Presentation Layer (Section 15.2)	Quantifies the value of a specific intervention before it is taken, and logs it for future validation